KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Verslag: getting acquainted with GPU programming

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste
Master in de ingenieurswetenschappen
Academiejaar 2024-2025

VERSLAG

Context

Het doel van dit labo was om kennis te maken met de absolute basis van GPU programming. Om dit te doen hebben wij een eenvoudige opgave opgelost: een array flippen in parallel met behulp van de GPU.

Open vragen

- Wat is de hoogst mogelijke block size?: **1024 threads per block**

Methodologie

Het flippen zelf doen wij met één thread per element. Elke GPU thread berekent de index in de 'source' array met $i = \text{blockIndex} * \text{threadsPerBlock} + \text{threadId}$. Na een simpele bounds check wordt het element in de source op de i-de plaats gezet op de $N - i - 1$-de plaats in de 'dest' array, met N als de grootte van de array.

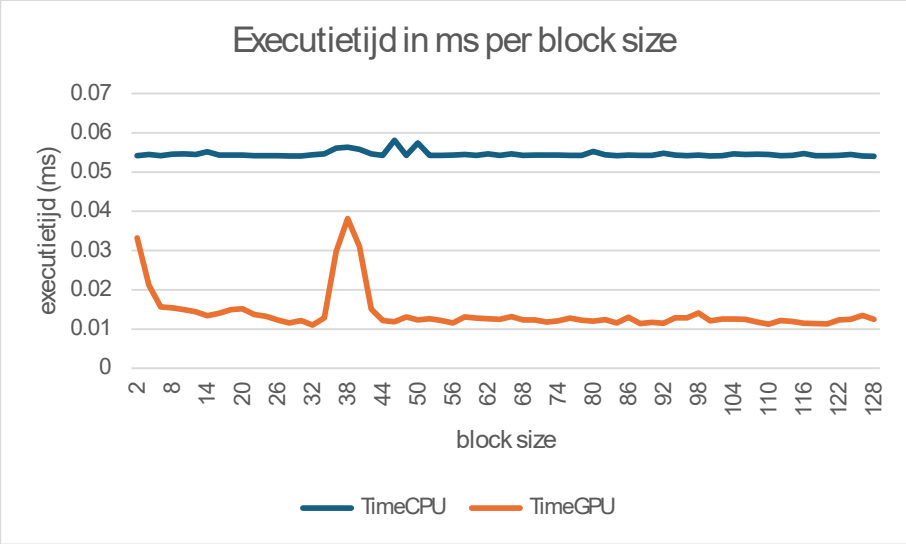
De hoeveelheid nodige blocks berekenen we afhankelijk van de block size en de grootte van de te flippen array. Deze hoeveelheid wordt als volgt berekent: $(N + T - 1)/T$ met N de grootte van de array en T het aantal threads.

Observaties en resultaten

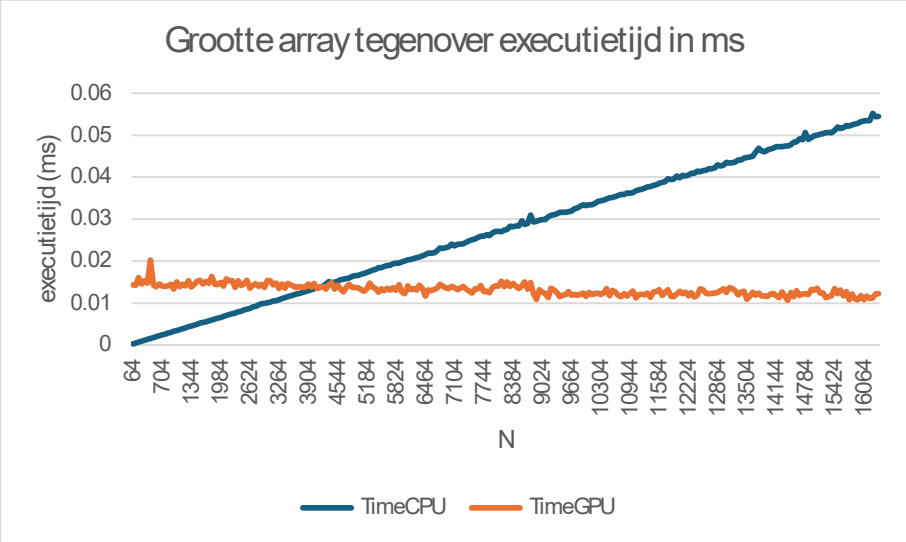
Wij hebben aan de bovenstaande implementatie timing-informatie toegevoegd. Wij houden de tijd dat de GPU nodig heeft om dit uit te voeren bij, de tijd op de CPU om de data te verplaatsen en de operatie op de gpu uit te voeren en een referentietijd van een puur CPU implementatie van dit algoritme met een for-loop.

In Figuur 1 worden de resultaten van ons éérste experiment weergegeven: De invloed van block size op de uitvoeringstijd. We zien dat naarmate de block size stijgt, de uitvoeringstijd ook lager wordt. Zoals te verwachten stopt dit effect echter wel op een gegeven moment. Het moment waarop deze limiet wordt bereikt, ligt ongeveer bij het aantal cores van de SM's (in dit geval 64). Het heeft dus op de meeste GPU's weinig nut om tot de maximale block size (1024) te gaan.

In Figuur 2 worden de resultaten van ons tweede experiment weergegeven: Het verschil tussen de uitvoeringstijden op de CPU en GPU in functie van de grootte van de array. Hierin is te zien dat de uitvoeringstijd op de GPU vrij gelijk blijft. De uitvoeringstijd op de CPU stijgt echter lineair. Dit is zoals verwacht, aangezien op de GPU, naarmate de array groter wordt, simpelweg meer threads kunnen worden toegevoegd die parallel werken.



Figuur 1: Block size tegenover executietijd in ms



Figuur 2: Array grootte tegenover executietijd in ms

Het is pas wanneer de array zo groot zou worden dat er niet genoeg threads zijn om alles in één keer uit te voeren, dat de tijd zou toenemen.

Conlcusie

Wij hebben succesvol een eerste stap gezet in GPU programming door middel van het flippen van een array. Wij hebben hier experimenten op gedaan om de effecten van de block size en de grootte van de array te meten. Hierbij komen wij de verwachtte resultaten uit: Heel lage block sizes hebben een negatief effect en verbeteringen stoppen vanaf het aantal cores van de SM's bereikt zijn. De uitvoeringstijd bij grotere arrays op de GPU stijgt ook niet door de parallellisatie en stijgt wel bij de CPU.

A CODE

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <chrono>


#define N 65536/sizeof(int)
#define THREADS_PER_BLOCK 64
#define AMOUNT_OF_BLOCKS ((N + THREADS_PER_BLOCK - 1) /
↪  THREADS_PER_BLOCK)


#define ITERATIONS 1000

struct TimeInfo {
    float timeCPU;
    float timeGPU;
};


struct BigTimeInfo {
    float timeGPUGPU;
    float timeGPUCPU;
    float timeCPU;
};


__global__ void flip__gpu(int *a, int *b, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // Global thread
    ↪  index

    if (i < N) {  // Prevent out-of-bounds access
        b[n - i - 1] = a[i];
    }
}


// both fn return the ms elapsed
float cpuFlip(int*a, int*b, int n) {
    // start time
```

```
    const auto start = std::chrono::steady_clock::now();
    for (int i = 0; i < n; i++) {
        b[n - i - 1] = a[i];
    }
    // end time
    const auto end = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::milli>(end -
    ↪  start).count();  // Convert to milliseconds
}

TimeInfo gpuFlip(int*a , int*b, int n, int threadsPerBlock, int
↪  amountOfBlocks) {
    TimeInfo timeInfo;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
        // Allocate memory on device
    int *dev_a, *dev_b;
    cudaMalloc((void**)&dev_a, n * sizeof(int));
    cudaMalloc((void**)&dev_b, n * sizeof(int));
    // start and end of CPU using chrona
    const auto startCPU = std::chrono::steady_clock::now();

    // Copy input array to device
    cudaMemcpy(dev_a, a, n * sizeof(int), cudaMemcpyHostToDevice);
    cudaEventRecord(start);

    // Configure kernel launch parameters
    // Launch the kernel with multiple blocks and threads
    flip__gpu<<<amountOfBlocks, threadsPerBlock>>>(dev_a, dev_b, n);
    cudaDeviceSynchronize(); // Ensure kernel execution completes
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    // Copy result back to host
    cudaMemcpy(b, dev_b, n * sizeof(int), cudaMemcpyDeviceToHost);


    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    timeInfo.timeGPU = milliseconds;
    timeInfo.timeCPU = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();  // Convert to milliseconds

     // Free device memory
    cudaFree(dev_a);
    cudaFree(dev_b);
    return timeInfo;
```

```
}


BigTimeInfo timedIteration(int n, int threadsPerBlock, int
↪  amountOfBlocks) {
    BigTimeInfo bigTimeInfo;
    int a[n], b[n], c[n];
    // Initialize input array
    for (int i = 0; i < n; i++) {
        a[i] = i;
    }

    TimeInfo gpuInfo = gpuFlip(a, b, n, threadsPerBlock, amountOfBlocks);
    float cpuMs = cpuFlip(a, c, n);
    for (int i = 0; i < n; i++) {
        if (b[i] != c[i]) {
            printf("Error: CPU and GPU results do not match\n");
            break;
        }
    }
    bigTimeInfo.timeGPUCPU = gpuInfo.timeCPU;
    bigTimeInfo.timeGPUGPU = gpuInfo.timeGPU;
    bigTimeInfo.timeCPU = cpuMs;
    return bigTimeInfo;
}

BigTimeInfo test(int n, int threadsPerBlock, int amountOfBlocks) {

    float cpuSum = 0;
    float gpuGpuSum = 0;
    float gpuCpuSum = 0;

    for (int i = 0; i < ITERATIONS; i++) {
        BigTimeInfo bigTimeInfo = timedIteration(n, threadsPerBlock,
        ↪  amountOfBlocks);
        cpuSum += bigTimeInfo.timeCPU;
        gpuGpuSum += bigTimeInfo.timeGPUGPU;
        gpuCpuSum += bigTimeInfo.timeGPUCPU;
    }

    BigTimeInfo bigTimeInfo;
    bigTimeInfo.timeCPU = cpuSum / ITERATIONS;
    bigTimeInfo.timeGPUGPU = gpuGpuSum / ITERATIONS;
    bigTimeInfo.timeGPUCPU = gpuCpuSum / ITERATIONS;
    return bigTimeInfo;
}
```

```
int main() {
    //printf("ThreadsPerBlock, TimeCPU, TimeGPU\n");
    /*
    for (int i = 2; i <= 128; i+= 2) {
        int threadsPerBlock = i;
        int amountOfBlocks = ((N + threadsPerBlock - 1) /
        ↪  threadsPerBlock);
        BigTimeInfo info = test(N, threadsPerBlock, amountOfBlocks);
        printf("%d, %f, %f\n", threadsPerBlock, info.timeCPU,
        ↪  info.timeGPUGPU);

    }
    */
    printf("N, TimeCPU, TimeGPU\n");
   for (int i = 64; i <= N; i+= 64) {
    int n = i;
    int amountOfBlocks = ((n + THREADS_PER_BLOCK - 1) /
    ↪  THREADS_PER_BLOCK);
    BigTimeInfo info = test(n, THREADS_PER_BLOCK, amountOfBlocks);
    printf("%d, %f, %f\n", n, info.timeCPU, info.timeGPUGPU);
   }
   return 0;
}
```